

Java Viva Questions:

Concise, viva-ready answers covering Java fundamentals through exception handling.

A. Java Fundamentals (Q1–20)

1. What is Java? Java is a high-level, object-oriented, class-based, general-purpose programming language created by James Gosling at Sun Microsystems (1995), now owned by Oracle. Its motto is "Write Once, Run Anywhere."

2. What are the main features of Java? Simple, object-oriented, platform independent, secure, robust (strong type checking, exception handling, garbage collection), portable, multithreaded, high performance (JIT compiler), distributed and dynamic.

3. Why is Java called platform independent? The compiler converts source code into bytecode, not native machine code. This bytecode runs on any operating system that has a JVM. The JVM is platform specific, but the bytecode is not.

4. What is the difference between source code, bytecode, and machine code?

- **Source code:** human-readable code written by the programmer (.java file).
- **Bytecode:** intermediate, platform-neutral code produced by javac (.class file).
- **Machine code:** binary instructions that the CPU executes directly; the JVM produces it from bytecode.

5. What is the JVM? The Java Virtual Machine is the engine that loads, verifies and executes bytecode. It contains the class loader, bytecode verifier, interpreter, JIT compiler and garbage collector.

6. What is the JDK? The Java Development Kit is the full toolkit for developing Java programs. It contains the JRE plus development tools such as javac, jar, javadoc and the debugger.

7. What is the JRE? The Java Runtime Environment provides what is needed to run Java programs: the JVM plus the core class libraries. It has no compiler.

8. What is the difference between JDK, JRE, and JVM?

Term	Purpose	Contains
JVM	Executes bytecode	Class loader, interpreter, JIT, GC
JRE	Runs Java programs	JVM + libraries
JDK	Develops and runs programs	JRE + compiler and tools

In short: $JDK \supset JRE \supset JVM$.

9. What is bytecode? Bytecode is the platform-independent, intermediate instruction set stored in .class files. The JVM interprets it or compiles it to native code with the JIT compiler.

10. Why is Java considered an object-oriented programming language? Everything is written inside classes, and it supports the four OOP pillars: encapsulation, inheritance, polymorphism and abstraction. (It is not 100% pure OO because primitive types are not objects.)

11. What is the role of the main() method in Java? `main()` is the entry point. The JVM calls it first to start program execution.

```
public static void main(String[] args) { }
```

12. Why is main() declared as public static void?

- **public:** so the JVM can access it from outside the class.
- **static:** so the JVM can call it without creating an object.
- **void:** it returns nothing to the JVM.

13. Can we execute a Java program without a main() method? No. In modern Java (7 and later) the program fails with an error saying the main method was not found. In Java 6 and earlier, a `static` block could run before the error. (Very recent Java versions have preview features for simplified entry points, but a standard program needs `main()`.)

14. What is a Java class? A class is a blueprint or template that defines the data (fields) and behavior (methods) of objects.

15. What is an object? An object is an instance of a class. It has state (field values), behavior (methods) and identity, and it is created with `new`.

16. What is the difference between a class and an object? A class is a logical template with no memory allocated for instance data. An object is a real instance of that class and occupies heap memory. One class can have many objects.

17. What is a reference variable? A variable that stores the address (reference) of an object rather than the object itself.

```
Student s = new Student(); // s is a reference variable
```

18. What is the difference between primitive data types and reference data types?

Primitive	Reference
Store actual values	Store address of an object
Fixed size, 8 types	Size depends on object
Default values like 0, false	Default value is null
Examples: <code>int</code> , <code>char</code>	Examples: <code>String</code> , arrays, classes

19. What are the eight primitive data types in Java? `byte` (1 byte), `short` (2), `int` (4), `long` (8), `float` (4), `double` (8), `char` (2), `boolean` (true/false).

20. What is type casting? Converting a value from one data type to another.

- **Widening (implicit):** smaller to larger type, e.g. `int` to `double`.
- **Narrowing (explicit):** larger to smaller type, may lose data.

```
int a = 10;
double d = a;           // widening
double x = 9.7;
int y = (int) x;        // narrowing, y = 9
```

B. Variables, Operators and Control Statements (Q21–36)

21. What is a local variable? A variable declared inside a method, constructor or block. It exists only while that block runs, has no default value and must be initialized before use.

22. What is an instance variable? A variable declared inside a class but outside methods, without `static`. Each object gets its own copy, stored in the heap, with default values.

23. What is a static variable? A class-level variable declared with `static`. Only one copy exists and it is shared by all objects of the class.

24. What is the difference between `==` and `=`? `=` is the assignment operator (stores a value). `==` is the equality comparison operator (returns true or false).

25. What is the difference between `==` and `.equals()`? `==` compares primitive values, or the references (addresses) of objects. `.equals()` compares the content of objects when it is overridden, as in `String`.

```
String a = new String("hi");
String b = new String("hi");
System.out.println(a == b);           // false
System.out.println(a.equals(b));      // true
```

26. What are relational operators in Java? Operators that compare two values and return a boolean: `==`, `!=`, `>`, `<`, `>=`, `<=`.

27. What are logical operators? Operators that combine boolean conditions: `&&` (AND), `||` (OR), `!` (NOT).

28. What is the difference between `&&` and `&`? `&&` is short-circuit: if the left side is false, the right side is not evaluated. `&` (when used with booleans) always evaluates both sides. `&` is also a bitwise operator on integers.

29. What is the ternary operator? A shorthand for if-else with three operands: `condition ? valueIfTrue : valueIfFalse`.

```
int max = (a > b) ? a : b;
```

30. What is the difference between `break` and `continue`? `break` terminates the loop (or switch) completely. `continue` skips the rest of the current iteration and moves on to the next one.

31. What is the difference between `while` and `do-while`? `while` checks the condition first (entry-controlled), so it may run zero times. `do-while` checks after the body (exit-controlled), so it runs at least once.

32. When would you use a `for` loop? When the number of iterations is known in advance, e.g. traversing an array or counting from 1 to n.

33. What is a nested loop? A loop placed inside another loop. The inner loop completes all its iterations for every single iteration of the outer loop (e.g. printing patterns, matrices).

34. What is a switch statement? A multi-way branching statement that compares one variable against several constant case values and runs the matching block, with an optional default.

35. Can a switch statement work with a String? Yes, since Java 7. It also works with byte, short, char, int, enums and wrapper types, but not with long, float, double or boolean.

36. What happens if break is omitted from a switch case? Execution "falls through" into the following cases until a break or the end of the switch is reached.

C. Arrays and Strings (Q37–55)

37. What is an array? A container object that holds a fixed number of elements of the same type in contiguous memory, accessed by index starting from 0.

38. Why are arrays called fixed-size data structures? Because the size is set at creation and cannot change afterward. To grow, a new array must be created and the data copied.

39. How do you declare and initialize an array in Java?

```
int[] a = new int[5];           // size 5, default values
int[] b = {10, 20, 30};        // with values
int[] c = new int[]{1, 2, 3};
```

40. What is the default value of elements in an integer array? 0.

41. How do you find the length of an array? Using the length property (no parentheses): arr.length.

42. What happens when you access an invalid array index? The JVM throws `ArrayIndexOutOfBoundsException` at runtime. A negative size throws `NegativeArraySizeException`.

43. What is a two-dimensional array? An array of arrays, used to represent rows and columns (a matrix).

```
int[][] m = {{1, 2, 3}, {4, 5, 6}};
System.out.println(m[1][2]); // 6
```

44. What is the difference between an array and an ArrayList?

Array	ArrayList
Fixed size	Dynamic size
Holds primitives and objects	Holds objects only (uses wrappers)
Uses length	Uses size()
Part of the language	Part of the Collections Framework

45. What is a String in Java? A String is an object of the `java.lang.String` class that represents a sequence of characters. String literals are stored in the String Constant Pool.

46. Why is String immutable? Once created, its content cannot change. This gives security (e.g. file paths, passwords), thread safety, caching of the hash code, and allows the String pool to share objects safely.

47. What is the difference between String, StringBuilder, and StringBuffer?

	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Speed	Slow for edits	Fastest	Slower than StringBuilder

48. What is the difference between equals() and equalsIgnoreCase()? equals() compares content and is case-sensitive. `equalsIgnoreCase()` compares content ignoring case.

```
"Java".equals("java");           // false
"Java".equalsIgnoreCase("java"); // true
```

49. What is the use of length() in a String? Returns the number of characters in the string:

```
"Hello".length() is 5.
```

50. What is the use of substring()? Extracts part of a string. `substring(begin)` goes to the end; `substring(begin, end)` includes begin and excludes end.

```
"Hello World".substring(6); // World
"Hello World".substring(0, 5); // Hello
```

51. What is the use of charAt()? Returns the character at a given index: `"Java".charAt(2)` is 'v'. An invalid index throws `StringIndexOutOfBoundsException`.

52. What is the difference between trim() and strip()? Both remove leading and trailing whitespace. `trim()` removes only characters up to space (ASCII `\u0020` and below); `strip()` (Java 11+) is Unicode-aware and removes all Unicode whitespace. `stripLeading()` and `stripTrailing()` are also available.

53. What do toUpperCase() and toLowerCase() do? They return a new string with all characters converted to upper or lower case. The original string is unchanged because Strings are immutable.

54. What is the use of contains() in String? Checks whether a string contains a given sequence of characters and returns `true` or `false`: `"Hello".contains("ell")` is `true`.

55. What is the difference between concat() and the + operator for Strings? `concat()` accepts only a String argument (a null argument throws `NullPointerException`) and returns a new String. The `+` operator can join Strings with any type (int, char, objects), converting them automatically, and treats null as the text "null".

D. OOP Concepts (Q56–80)

56. What are the four main principles of OOP? Encapsulation, inheritance, polymorphism and abstraction.

57. What is encapsulation? Binding data (fields) and the methods that operate on it into a single unit (class), and restricting direct access to the data. It is also called data hiding.

58. How is encapsulation achieved in Java? Declare fields `private` and provide public getter and setter methods.

```

class Student {
    private int age;
    public int getAge() { return age; }
    public void setAge(int age) {
        if (age > 0) this.age = age;
    }
}

```

59. What is inheritance? The mechanism by which one class (subclass/child) acquires the fields and methods of another class (superclass/parent) using `extends`. It models an IS-A relationship.

60. Why is inheritance used? For code reusability, to build a hierarchy of related classes, and to support runtime polymorphism (method overriding).

61. What is the difference between `extends` and `implements`? `extends` is used by a class to inherit from one class (or by an interface to inherit from interfaces). `implements` is used by a class to implement one or more interfaces.

62. What is polymorphism? The ability of one entity to take many forms: the same method name or reference can behave differently depending on context. Java has compile-time and runtime polymorphism.

63. What is compile-time polymorphism? Polymorphism resolved by the compiler, achieved through method overloading (static binding).

64. What is runtime polymorphism? Polymorphism resolved by the JVM at runtime, achieved through method overriding where a parent reference points to a child object (dynamic binding).

```

Animal a = new Dog();
a.sound(); // Dog's version runs

```

65. What is method overloading? Having multiple methods with the same name in the same class but different parameter lists (number, type or order of parameters).

66. What is method overriding? A subclass provides its own implementation of a method already defined in its superclass, with the same name, parameters and compatible return type.

67. What is the difference between method overloading and overriding?

Overloading	Overriding
Same class	Parent and child classes
Parameters must differ	Parameters must be the same
Compile-time polymorphism	Runtime polymorphism
Return type may differ	Return type same or covariant
Can overload static methods	Cannot override static, final, private methods

68. Can we overload a method by changing only its return type? No. The compiler cannot distinguish the methods by return type alone, so it gives a compile-time error. The parameter list must differ.

69. What is the super keyword? `super` refers to the immediate parent class. It is used to access parent fields (`super.x`), call parent methods (`super.show()`) and call the parent constructor (`super(...)`, which must be the first statement).

70. What is the this keyword? `this` refers to the current object. It is used to distinguish instance variables from parameters (`this.age = age`), call another constructor (`this(...)`), or pass the current object.

71. What is a constructor? A special method that has the same name as the class and no return type. It is called automatically when an object is created with `new` and is used to initialize the object.

72. What is the difference between a constructor and a method?

Constructor	Method
Same name as class	Any valid name
No return type	Must have a return type (or void)
Called automatically by new	Called explicitly
Initializes an object	Defines behavior
Not inherited	Inherited

73. Can a constructor be overloaded? Yes. A class can have multiple constructors with different parameter lists.

74. Can a constructor be inherited? No. Constructors are not inherited, but a subclass constructor can call the parent constructor using `super()`.

75. What is an abstract class? A class declared with the `abstract` keyword that cannot be instantiated. It may contain abstract methods (without a body) and concrete methods, and is meant to be extended.

76. What is an interface? A reference type that defines a contract: a set of abstract methods (plus constants, and default/static methods since Java 8) that implementing classes must provide.

77. Why do we use interfaces? To achieve full abstraction, loose coupling, multiple inheritance of type, and a common standard that unrelated classes can follow.

78. What is the difference between an abstract class and an interface?

Abstract class	Interface
Uses <code>extends</code>	Uses <code>implements</code>
Single inheritance only	A class can implement many
Can have constructors and instance variables	No constructors; variables are <code>public static final</code>
Abstract and concrete methods	Abstract, default and static methods

Abstract class	Interface
Any access modifiers	Methods are public by default

79. Can a class implement multiple interfaces? Yes. This is how Java achieves multiple inheritance of type: `class A implements X, Y { }`.

80. Can an interface extend another interface? Yes. An interface can extend one or more interfaces: `interface C extends A, B { }`.

E. Access Modifiers, Packages and Core Concepts (Q81–90)

81. What are the access modifiers in Java? `private`, default (no keyword, package-private), `protected` and `public`.

82. What is the difference between private, default, protected, and public?

Modifier	Same class	Same package	Subclass (other package)	Other packages
<code>private</code>	Yes	No	No	No
default	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes	No
<code>public</code>	Yes	Yes	Yes	Yes

83. Why are data members generally declared private? To achieve encapsulation: it protects data from unwanted modification, allows validation in setters, and lets the internal implementation change without affecting other code.

84. What is a package? A namespace that groups related classes, interfaces and sub-packages, e.g. `java.util`, `java.io`.

85. Why are packages used in Java? To organize code, avoid name conflicts, control access (default and protected access) and make classes easier to find and reuse.

86. What is the difference between import and package? `package` declares which package the current class belongs to (it must be the first statement). `import` makes classes from other packages available so they can be used without the full name.

```
package com.college.app;
import java.util.Scanner;
```

87. What is the purpose of the static keyword? `static` makes a member belong to the class rather than to any object. It applies to variables (shared copy), methods (callable without an object), blocks (run once at class loading) and nested classes. Static methods cannot directly use instance members or `this`.

88. What is the purpose of the final keyword? `final` restricts modification:

- `final` variable: value cannot be changed (a constant).
- `final` method: cannot be overridden.

- `final` class: cannot be inherited (e.g. `String`).

89. Can a final variable be modified? No, it cannot be reassigned once initialized. For a final reference variable, the reference cannot change, but the object's own content can still be modified.

90. Can a final method be overridden? No. A subclass cannot override a `final` method, though it can inherit and call it.

F. Exception Handling (Q91–100)

91. What is an exception? An unwanted event that occurs at runtime and disrupts the normal flow of the program, e.g. dividing by zero or accessing a null reference.

92. What is exception handling? The mechanism of detecting and responding to exceptions using `try`, `catch`, `finally`, `throw` and `throws`, so the program continues or ends gracefully.

93. What is the difference between an error and an exception? An **Error** (e.g. `OutOfMemoryError`, `StackOverflowError`) indicates a serious system-level problem that applications should not try to handle. An **Exception** is a condition the program can anticipate and handle. Both extend `Throwable`.

94. What is the difference between checked and unchecked exceptions?

Checked	Unchecked
Checked by compiler at compile time	Occur at runtime, not checked by compiler
Must be handled or declared with <code>throws</code>	Handling is optional
Examples: <code>IOException</code> , <code>SQLException</code>	Examples: <code>NullPointerException</code> , <code>ArithmeticException</code>
Subclass of <code>Exception</code> (not <code>RuntimeException</code>)	Subclass of <code>RuntimeException</code>

95. What is the purpose of try and catch? The `try` block holds code that may throw an exception. The `catch` block handles a specific exception type when it occurs.

```
try {
    int r = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

96. What is the purpose of the finally block? Code in `finally` always executes whether or not an exception occurs, so it is used for cleanup such as closing files and connections. The only exceptions are `System.exit()` or a JVM crash.

97. What is the difference between throw and throws?

<code>throw</code>	<code>throws</code>
Used to explicitly throw an exception object	Declares exceptions a method may throw
Used inside a method body	Used in the method signature

throw	throws
Throws one exception at a time	Can list multiple exceptions

```
void check(int age) throws Exception {
    if (age < 18) throw new Exception("Not eligible");
}
```

98. Can we create our own exception class? Yes, by extending Exception (checked) or RuntimeException (unchecked). These are called user-defined or custom exceptions.

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) { super(msg); }
}
```

99. What happens if an exception is not handled? The JVM's default handler prints the exception name, message and stack trace, then terminates the program abnormally (the current thread).

100. Why should exceptions be handled instead of allowing the program to terminate unexpectedly?

Handling keeps the program running or lets it end gracefully, separates error-handling code from normal logic, gives meaningful messages to users, allows cleanup of resources, and prevents data loss or corruption.